DOCUMENTACIÓN OFICIAL · JURETRAVEL

Guía de Uso y Funcionamiento — JureFinance

Sistema integral de cotización de viajes, arbitraje de proveedores, emisión de aéreos, liquidación por bloques (BCP) y sincronización con Google Drive.

1. Idea General y Propósito del Proyecto

JureFinance es la aplicación web de gestión operativa, financiera y documental diseñada a medida para la agencia de viajes **JureTravel**.

Su objetivo principal es **centralizar los números reales de la agencia** en un único entorno digital, cubriendo de punta a punta el circuito operativo diario: **cotización y arbitraje de proveedores → venta al pasajero → reserva y emisión de aéreos → liquidación por bloques (BCP) y pago a operadores mayoristas (TTOO) → cobranzas y expedientes digitales en Google Drive**.

> **Concepto clave de negocio:** El presupuesto comercial que recibe el viajero (PDF con markup incluido) se genera fuera de la aplicación con la herramienta del proveedor mayorista. **JureFinance actúa como el libro mayor interno de la agencia**, donde se registra cuánto paga realmente el cliente (`precioVenta`) frente a cuánto le cuesta realmente a la agencia cada servicio (`costo` de la oferta de compra elegida), permitiendo vender con el presupuesto de un proveedor y comprar el servicio a otro más económico o bonificado.

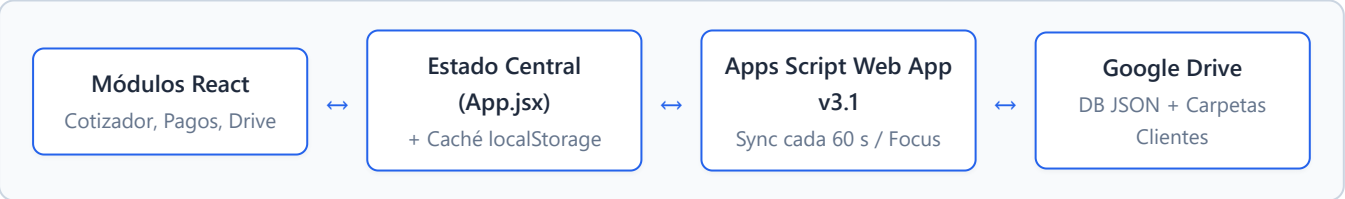
¿Para qué sirve?

1. **Registrar cotizaciones y calcular la ganancia real por servicio:** Permite comparar ofertas de múltiples operadores mayoristas (Aero, PriceTravel, Azabache, etc.) para un mismo servicio, elegir con cuál se compra efectivamente y obtener el margen real neto ($Ganancia = \Sigma\ Precio\ de\ Venta - \Sigma\ Costo\ de\ Compra$).
2. **Controlar el ciclo de emisión de aéreos:** Lleva la trazabilidad de cada vuelo (`SinReservar` → `Reservado` → `Emitido`) y calcula automáticamente cuándo vence el pago al operador según la fecha de emisión.
3. **Automatizar los bloques de pago (BCP) por proveedor:** Modela la operatoria real de consolidadores como **AERO**, donde los boletos emitidos dentro de un rango de días (p. ej., del 1 al 5 del mes) se acumulan en una "caja" y se liquidan todos juntos en una fecha de pago determinada (p. ej., el día 20).
4. **Gestionar Cuentas a Pagar y Flujo de Caja Auditable:** Agrupa obligaciones individuales y liquidaciones consolidadas por proveedor, fecha de vencimiento y moneda (`USD` / `ARS`), permitiendo registrar pagos parciales, individuales o en bloque (reparto automático **FIFO**) y exportar el reporte mensual a **CSV**.
5. **Seguir Cuentas a Cobrar y Métricas (Dashboard):** Organiza los cobros a clientes por mes y presenta indicadores ejecutivos de rentabilidad, ventas cerradas y alertas de vencimientos próximos.
6. **Sincronizar Expedientes y Base de Datos con Google Drive:** Conecta en tiempo real la aplicación con las carpetas de clientes en Google Drive (vouchers, pasajes, DNI/pasaportes) y respalda toda la base operativa mediante un conector propio en **Google Apps Script (v3.1)**.

2. Arquitectura y Stack Tecnológico

La aplicación está construida como una **Single Page Application (SPA)** con persistencia híbrida (**Local-First + Sincronización Bidireccional con Google Drive** vía Google Apps Script).

Capa	Tecnología	Propósito
Frontend UI	React 19 + Vite 8	Interfaz reactiva modular con estado centralizado en `src/App.jsx`.
Estilos y Diseño	CSS Variables + Lucide React	Sistema de diseño limpio, adaptable y con iconografía semántica.
Lógica de Negocio	ES Modules (`src/services/`)	Motores puros de cálculo para servicios, bloques TTOO, liquidaciones FIFO y flujo de caja.
Persistencia Local	`localStorage`	Caché inmediata en el navegador para operación sin latencia.
Backend / Cloud Sync	Google Apps Script (`v3.1`)	Conector serverless (`google-apps-script/Code.gs`) que sincroniza `jurefinance_db.json` y el árbol precalculado `jurefinance_tree.json` en Drive.
Calidad y Testing	Vite SSR + Node + Oxlint	Suite de pruebas del modelo matemático (`test:modelo`), vistas SSR (`test:ui`) y navegador real (`test:runtime`).



3. Circuito Operativo de la Agencia

1. **Configuración de Proveedores y Calendario Mensual (BCP):** Se dan de alta los operadores mayoristas (**Aero**, **Price**, **Azabache**, etc.) y en **Configuración Mensual** se definen para cada proveedor y mes los bloques de días de emisión y su fecha de pago asociada (manualmente, con plantilla estándar, por regla automática o copiando del mes anterior).
2. **Carga de la Cotización (Wizard de 3 pasos):**
   - **Paso 1 (Datos del viaje):** Cliente titular, nombres de pasajeros (`pasajeros[]`), destino, fechas del viaje, cantidad de PAX, moneda (`USD` o `ARS`) y proveedores participantes.
   - **Paso 2 (Servicios y costos):** Por cada servicio (Aéreo, Alojamiento, Traslado, Asistencia, etc.) se carga el **precio de venta total** informado al cliente y una o varias **ofertas de compra** de distintos proveedores, seleccionando con cuál se compra (`ofertaCompraId`).
   - **Paso 3 (Resumen y Guardado):** Cálculo automático de venta total, costo real, ganancia neta y margen porcentual, con **Resumen Interno Imprimible**.

3. **Seguimiento Comercial (Estados de Cotización):** La cotización avanza por los estados: `Borrador` → `Enviada` → `Aceptada` → `Pagada` (o `Vencida`).
4. **Reserva y Emisión de Aéreos:** En los servicios aéreos se controla el estado del boleto: `SinReservar` → `Reservado` → `Emitido` (con `fechaEmision`). Al emitir, si la oferta no tenía vencimiento manual, el sistema asigna automáticamente la fecha de pago del bloque BCP correspondiente.
5. **Liquidación y Pago a Proveedores (Cuentas a Pagar):** Las ofertas de compra con costo mayor a `0` generan obligaciones en el calendario. En **Liquidación por proveedor / Flujo de Caja BCP**, los boletos emitidos en un mismo bloque aparecen consolidados bajo una única fecha de vencimiento. Se puede abonar un ticket individual o registrar un **pago consolidado del bloque** (reparto automático **FIFO**).
6. **Cobranza al Cliente y Expediente en Drive:** Se monitorean los cobros mensuales derivados de las ventas y se archivan en la carpeta de Google Drive del cliente los vouchers, pasajes emitidos y copias de DNI/pasaportes.

4. Guía de Uso por Módulo

4.1. Cotizador (`src/components/Cotizador/`)

Es el corazón operativo para presupuestar y registrar ventas mediante un Wizard de 3 pasos (`CotizadorWizard.jsx`):

- **Paso 1 — Datos del Viaje:** Cliente, contacto, destino, fechas, cantidad de pasajeros, lista nominal de viajeros y selección de operadores participantes.
- **Paso 2 — Servicios y Costos:** Permite agregar múltiples renglones (`Aéreo` , `Hotel` , `Traslado` , `Asistencia` , `Excursión` , `Crucero` , `Otro`), cargar el `precioVenta` total del grupo y comparar `ofertas[]` de distintos proveedores. En servicios **Aéreos** habilita ruta, aerolínea y estado (`SinReservar` , `Reservado` , `Emitido`).
- **Paso 3 — Resumen y Guardado:** Desglose financiero final, cambio de estado comercial y modal de **Resumen Interno** listo para imprimir.

4.2. Historial de Cotizaciones (`src/components/Historial/HistorialView.jsx`)

Listado completo de cotizaciones guardadas con buscador por cliente, destino o ID y filtros por estado comercial. Permite **Editar**, **Duplicar**, **Abrir Carpeta en Drive** y **Eliminar**.

4.3. Configuración Mensual BCP (`src/components/Configuracion/ConfiguracionMensualView.jsx`)

Administra las reglas de calendario ("día de emisión → fecha de pago") por operador y mes (`YYYY-MM`):

- **Edición manual de bloques:** Define rangos `DiaInicio` – `DiaFin` y su `FechaPago` .
- **Generador automático, Plantilla estándar y Copia entre meses:** Crea bloques de *N* días o clona meses anteriores adaptando automáticamente meses de 28, 29, 30 o 31 días.
- **Indicador de cobertura:** Verifica que todos los días del mes estén cubiertos sin huecos ni solapamientos.

4.4. Cuentas a Pagar a Proveedores (`CalendarioPagosView.jsx` y `FlujoCajaBCP.jsx`)

- **Calendario Mensual de Vencimientos:** Grilla visual día por día con estados (`Pendiente` , `Parcial` , `Pagado` , `Vencido`), filtros por proveedor y carga de compromisos manuales.
- **Liquidación por Proveedor (Pago en Bloque):** Agrupa obligaciones por `proveedor + fecha de vencimiento + moneda` y reparte pagos totales o parciales en orden **FIFO**.
- **Reporte de Flujo de Caja Auditable:** Totales mensuales separados estrictamente por moneda (`USD` / `ARS`), detalle auditable por ticket (cliente, pasajeros, PAX, destino, emisión y bloque BCP), **Exportación a CSV** y carga de 10 vuelos de demostración.

4.5. Cuentas a Cobrar, Operadores, Dashboard y Expedientes en Drive

- **Cuentas a Cobrar (`CalendarioCobrosView.jsx`):** Proyección mensual de ingresos derivados del `precioVenta` de las ventas.
- **Directorio de Operadores (`OperadoresView.jsx`):** Catálogo de proveedores mayoristas que alimenta el Cotizador y la Configuración BCP.
- **Dashboard Ejecutivo (`DashboardView.jsx`):** KPIs de ventas totales, costos netos, ganancia real acumulada, margen promedio y alertas de vencimientos urgentes.
- **Expedientes en Google Drive (`DriveView.jsx`):** Explorador de carpetas de clientes en tiempo real, subida de archivos y configuración del conector Apps Script v3.1.

5. Modelo Matemático y Reglas de Negocio Clave

Concepto	Regla en el Sistema
Importes por Servicio	Tanto `precioVenta` como el `costo` de cada oferta representan el **total del servicio para todo el grupo de pasajeros (PAX)**, nunca un valor unitario por pasajero.
Ganancia Real	$Ganancia = \Sigma\, precioVenta - \Sigma\, costo(ofertaCompra)$. Las ofertas alternativas quedan registradas como referencia comparativa pero no afectan ni la ganancia ni los pagos.
Servicios Bonificados	Si la oferta de compra seleccionada tiene `costo = 0` , el servicio suma su `precioVenta` a la ganancia pero **no genera obligación de pago** en el calendario.
Overlay de Pagos (`pagosTTOO`)	Los pagos realizados **no mutan la cotización**: se almacenan por separado y se cruzan en cada render mediante la clave `${cotizacionId}::${itemId}` .
Separación de Monedas	Los cálculos de liquidaciones y totales mensuales de flujo de caja agrupan siempre por moneda (`USD` / `ARS`), evitando mezclar divisas distintas.

6. Estructura del Proyecto

```
JureFinance/
├── docs/
│   ├── FLUJO_CLIENTE.md          # Especificación detallada del circuito operativo de JureTravel
│   └── CONTEXTO_Y_ESTADO.md      # Estado técnico, decisiones de arquitectura y roadmap
├── google-apps-script/
│   ├── Code.gs                   # Código fuente del conector Web App v3.1 para Google Drive
│   └── DESPLIEGUE_v3.1.md        # Guía paso a paso para publicar el conector en Apps Script
├── scripts/
│   ├── probarModelo.mjs          # Suite de 65 asertos sobre servicios, bloques BCP y pagos FIFO
│   ├── probarUI.mjs              # Pruebas de renderizado SSR de las vistas financieras
│   └── probarRuntime.mjs         # Smoke test automatizado en navegador headless
├── src/
│   ├── components/               # Vistas por módulo (Cotizador, Cobranzas, Configuracion, Drive,
etc.)
│   ├── data/                     # Semilla inicial (initialData.js) y vuelos demo (demoVuelos.js)
│   ├── services/                 # Motores de negocio (servicios, tto, calendario, cobro, drive)
│   ├── App.jsx                   # Controlador principal de estado y sincronización
│   └── main.jsx                  # Punto de entrada React
├── GUIA_DE_USO_Y_FUNCIONAMIENTO.md  # Guía general en formato Markdown
├── GUIA_DE_USO_Y_FUNCIONAMIENTO.pdf # Guía general en formato PDF
└── package.json                  # Dependencias y scripts de ejecución
```

7. Instalación, Uso y Verificación

Requisitos previos: Node.js 20 o superior y npm.

```
# 1. Instalar dependencias
npm install

# 2. Iniciar servidor de desarrollo (http://localhost:5173)
npm run dev

# 3. Ejecutar pruebas del modelo de negocio y agenda de pagos (65 validaciones)
npm run test:modelo

# 4. Ejecutar pruebas de renderizado de interfaz (SSR) y linter
npm run test:ui
npm run lint

# 5. Compilar versión de producción (carpeta dist/)
npm run build
```

JureFinance · Sistema de Gestión Operativa y Financiera para JureTravel · Repositorio oficial: github.com/elopezgit/JureFinance